Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.01 Информатика и вычислительная техника

ОТЧЕТ

Лабораторная работа № 12

По дисциплине «Теория алгоритмов и структуры данных»

Вариант № 25

Выполнил: студент группы ИВТ-25-2б
Соловьев Тимофей Николаевич

Принял: Доц. Полякова О. А.

Пермь, 2026

1. Постановка задачи

```
Постановка задачи
Задача 1.
1. Создать ассоциативный контейнер.
2. Заполнить его элементами стандартного типа (тип указан в варианте).
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.
Задача 2.
1. Создать ассоциативный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.
Задача 3
1. Создать параметризированный класс, используя в качестве контейнера
ассоциативный контейнер.
2. Заполнить его элементами.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде методов параметризированного класса. Задача 1
1. Контейнер - multiset
2. Тип элементов - double
Задача 2
Тип элементов Pair (см. лабораторную работу No3).
Задача 3
Параметризированный класс – Список (см. лабораторную работу No7) Задание 3 Найти среднее
арифметическое и
добавить его в конец
контейнера Задание 4 Найти элементы
ключами из заданного
диапазона и удалить их
из контейнера Задание 5 К каждому элементу добавить
сумму минимального и
максимального элементов
контейнера
```

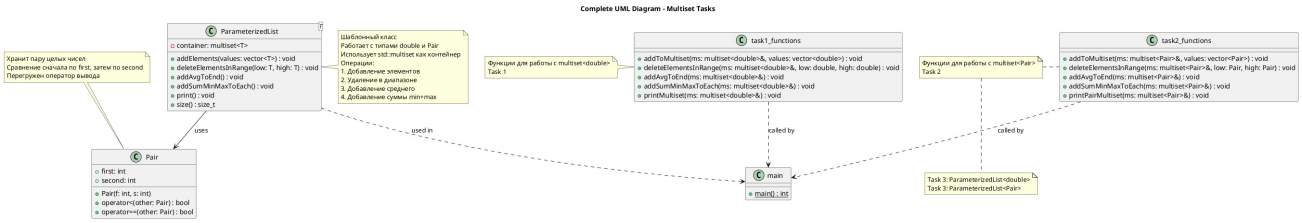
2. Анализ

```
Программа состоит из трех задач, демонстрирующих работу с ассоциативным контейнером multiset для
стандартных и пользовательских типов, а также параметризованного класса на его основе.

В первой задаче используется контейнер multiset для хранения вещественных чисел. Множество с повторениями
выбрано потому, что исходные данные содержат дублирующиеся значения, которые необходимо сохранить.
Функция добавления заполняет контейнер элементами из вектора с помощью метода insert. Для вычисления
среднего арифметического все элементы суммируются в цикле, после чего среднее значение добавляется в
контейнер. Удаление элементов в заданном диапазоне выполняется прямым обходом итератора с вызовом метода
erase, который возвращает следующий валидный итератор. Для добавления к каждому элементу суммы
минимального и максимального значений сначала находятся эти значения обращением к begin и rbegin, затем
создается новый контейнер, куда помещаются все элементы с увеличенными значениями, после чего исходный
контейнер заменяется новым. Все операции сопровождаются выводом промежуточных состояний.

Во второй задаче используется тот же контейнер multiset, но для хранения объектов пользовательского типа
Pair. Структура Pair содержит два целых поля. Оператор меньше перегружен так, что сравнение выполняется
сначала по первому полю, затем по второму. Это необходимо для определения порядка элементов в контейнере.
Оператор равенства перегружен для сравнения полей поэлементно. Оператор вывода выводит пару в круглых
скобках через запятую. Исходный список содержит шесть пар, включая повторяющиеся. Для вычисления среднего
арифметического суммируются отдельно первые поля и отдельно вторые, после чего создается новая пара из
целых частей от деления сумм на размер контейнера. Удаление элементов в заданном диапазоне осуществляется
аналогично первой задаче, но сравнение выполняется с использованием перегруженного оператора меньше. Для
добавления суммы минимальной и максимальной пар находятся первая и последняя пары в упорядоченном
контейнере, вычисляется новая пара, сложением соответствующих полей, затем каждый элемент увеличивается
на эти значения.

В третьей задаче создается шаблонный класс ParameterizedList, который внутри использует multiset.
Конструктор класса не требуется, так как элементы добавляются через метод addElements. Метод addAvgToEnd
использует вызов constexpr if для различения типов. Для типа double суммируются значения элементов,
среднее добавляется в контейнер. Для типа Pair суммируются отдельно первые и вторые поля, создается пара
из средних арифметических, которая также добавляется. Метод deleteElementsInRange удаляет все элементы,
```

которые находятся между low и high, используя тот же алгоритм обхода итератора. Метод addSumMinMaxToEach находит минимальный и максимальный элементы через begin и rbegin, вычисляет сумму минимального и максимального, затем для каждого типа по-разному обрабатывает добавление. Для double к каждому элементу прибавляется вычисленная сумма. Для Pair создается новая пара, где каждое поле увеличено на соответствующее поле суммы минимальной и максимальной пар. Метод print выводит все элементы контейнера.

Демонстрационная программа создает объект ParameterizedList для типа double, добавляет в него шесть значений, показывает начальное состояние. Затем добавляет среднее арифметическое, удаляет элементы в интервале от двух до пяти, и наконец прибавляет к каждому элементу сумму минимального и максимального. Аналогичные операции выполняются для ParameterizedList с типом Pair. Исходные данные содержат пять пар, выполняются добавление среднего, удаление пар в диапазоне между (2,3) и (6,7), а затем добавление суммы минимальной и максимальной пар к каждому элементу. После каждой операции выводится текущее содержимое контейнера для контроля корректности работы всех методов.

3. UML-диаграмма



4. Исходный код

Файл реализации (.cpp)

```
#include
#include
#include
#include
#include
#include

class Pair {
public:
    int first;
    int second;

    Pair(int f = 0, int s = 0) : first(f), second(s) {}

    bool operator<(const Pair& other) const {
        if (first != other.first) return first < other.first;
        return second < other.second;
    }

    bool operator==(const Pair& other) const {
        return first == other.first && second == other.second;
    }

    friend std::ostream& operator<<(std::ostream& os, const Pair& p) {
        os << "(" << p.first << "," << p.second << ")";
        return os;
    }
};

void task1_addToMultiset(std::multiset& ms, const std::vector& values) {
    for (double val : values) ms.insert(val);
}

void task1_deleteElementsInRange(std::multiset& ms, double low, double high) {
    auto it = ms.begin();
    while (it != ms.end()) {
        if (*it >= low && *it <= high) it = ms.erase(it);
        else ++it;
    }
}

void task1_addAvgToEnd(std::multiset& ms) {
    if (ms.empty()) return;
    double sum = 0;
    for (double val : ms) sum += val;
```

```
    double avg = sum / ms.size();
    ms.insert(avg);
}
void task1_addSumMinMaxToEach(std::multiset& ms) {
    if (ms.size() < 2) return;
    double minVal = *ms.begin();
    double maxVal = *ms.rbegin();
    double sumMinMax = minVal + maxVal;

    std::multiset newSet;
    for (double val : ms) newSet.insert(val + sumMinMax);
    ms = newSet;
}

void printMultiset(const std::multiset& ms) {
    for (double val : ms) std::cout << val << " ";
    std::cout << std::endl;
}

void task2_addToMultiset(std::multiset& ms, const std::vector& values) {
    for (const Pair& p : values) ms.insert(p);
}

void task2_deleteElementsInRange(std::multiset& ms, const Pair& low, const Pair& high) {
    auto it = ms.begin();
    while (it != ms.end()) {
        if (low < *it && *it < high) it = ms.erase(it);
        else ++it;
    }
}

void task2_addAvgToEnd(std::multiset& ms) {
    if (ms.empty()) return;
    int sumFirst = 0, sumSecond = 0;
    for (const Pair& p : ms) {
        sumFirst += p.first;
        sumSecond += p.second;
    }
    Pair avg(sumFirst / ms.size(), sumSecond / ms.size());
    ms.insert(avg);
}

void task2_addSumMinMaxToEach(std::multiset& ms) {
    if (ms.size() < 2) return;
    Pair minPair = *ms.begin();
    Pair maxPair = *ms.rbegin();
    Pair sumMinMax(minPair.first + maxPair.first, minPair.second + maxPair.second);

    std::multiset newSet;
    for (const Pair& p : ms) {
        newSet.insert(Pair(p.first + sumMinMax.first, p.second + sumMinMax.second));
    }
    ms = newSet;
}

void printPairMultiset(const std::multiset& ms) {
    for (const Pair& p : ms) std::cout << p << " ";
    std::cout << std::endl;
}

template
class ParameterizedList {
private:
    std::multiset container;

public:
    void addElements(const std::vector& values) {
        for (const T& val : values) container.insert(val);
    }

    void deleteElementsInRange(const T& low, const T& high) {
        auto it = container.begin();
        while (it != container.end()) {
            if (low < *it && *it < high) it = container.erase(it);
            else ++it;
        }
    }

    void addAvgToEnd() {
```

```
        if (container.empty()) return;

        if constexpr (std::is_same::value) {
            double sum = 0;
            for (double val : container) sum += val;
            double avg = sum / container.size();
            container.insert(avg);
        }
        else if constexpr (std::is_same::value) {
            int sumFirst = 0, sumSecond = 0;
            for (const Pair& p : container) {
                sumFirst += p.first;
                sumSecond += p.second;
            }
            Pair avg(sumFirst / container.size(), sumSecond / container.size());
            container.insert(avg);
        }
    }

    void addSumMinMaxToEach() {
        if (container.size() < 2) return;

        T minVal = *container.begin();
        T maxVal = *container.rbegin();

        if constexpr (std::is_same::value) {
            double sumMinMax = minVal + maxVal;
            std::multiset newSet;
            for (T val : container) newSet.insert(val + sumMinMax);
            container = newSet;
        }
        else if constexpr (std::is_same::value) {
            Pair sumMinMax(minVal.first + maxVal.first, minVal.second + maxVal.second);
            std::multiset newSet;
            for (const Pair& p : container) {
                newSet.insert(Pair(p.first + sumMinMax.first, p.second + sumMinMax.second));
            }
            container = newSet;
        }
    }

    void print() const {
        for (const T& val : container) std::cout << val << " ";
        std::cout << std::endl;
    }

    size_t size() const { return container.size(); }
};

int main() {
    std::cout << "========== TASK 1: multiset ==========" << std::endl;
    std::multiset ms1;
    std::vector init1 = {1.5, 2.3, 4.7, 5.1, 2.3, 3.9, 6.2, 1.5};
    task1_addToMultiset(ms1, init1);
    std::cout << "Initial multiset: "; printMultiset(ms1);

    task1_addAvgToEnd(ms1);
    std::cout << "After adding average to end: "; printMultiset(ms1);

    task1_deleteElementsInRange(ms1, 2.0, 5.0);
    std::cout << "After deleting elements in [2.0,5.0]: "; printMultiset(ms1);

    task1_addSumMinMaxToEach(ms1);
    std::cout << "After adding sum(min+max) to each: "; printMultiset(ms1);

    std::cout << "\n========== TASK 2: multiset ==========" << std::endl;
    std::multiset ms2;
    std::vector init2 = {Pair(1,2), Pair(3,4), Pair(5,6), Pair(1,2), Pair(7,8), Pair(3,4)};
    task2_addToMultiset(ms2, init2);
    std::cout << "Initial multiset: "; printPairMultiset(ms2);

    task2_addAvgToEnd(ms2);
    std::cout << "After adding average to end: "; printPairMultiset(ms2);

    task2_deleteElementsInRange(ms2, Pair(2,3), Pair(6,7));
    std::cout << "After deleting elements in range (2,3) to (6,7): "; printPairMultiset(ms2);

    task2_addSumMinMaxToEach(ms2);
    std::cout << "After adding sum(min+max) to each: "; printPairMultiset(ms2);
```

```
    std::cout << "\n========== TASK 3: ParameterizedList ==========" << std::endl;
    ParameterizedList list1;
    std::vector init3 = {1.5, 2.3, 4.7, 5.1, 2.3, 3.9};
    list1.addElements(init3);
    std::cout << "Initial list: "; list1.print();

    list1.addAvgToEnd();
    std::cout << "After adding average: "; list1.print();

    list1.deleteElementsInRange(2.0, 5.0);
    std::cout << "After deleting in (2.0,5.0): "; list1.print();

    list1.addSumMinMaxToEach();
    std::cout << "After adding sum(min+max): "; list1.print();

    std::cout << "\n========== TASK 3: ParameterizedList ==========" << std::endl;
    ParameterizedList list2;
    std::vector init4 = {Pair(1,2), Pair(3,4), Pair(5,6), Pair(1,2), Pair(7,8)};
    list2.addElements(init4);
    std::cout << "Initial list: "; list2.print();

    list2.addAvgToEnd();
    std::cout << "After adding average: "; list2.print();

    list2.deleteElementsInRange(Pair(2,3), Pair(6,7));
    std::cout << "After deleting in range: "; list2.print();

    list2.addSumMinMaxToEach();
    std::cout << "After adding sum(min+max): "; list2.print();

    return 0;
}
```

5. Результаты выполнения

```
========== TASK 1: multiset ==========
Initial multiset: 1.5 1.5 2.3 2.3 3.9 4.7 5.1 6.2
After adding average to end: 1.5 1.5 2.3 2.3 3.4375 3.9 4.7 5.1 6.2
After deleting elements in [2.0,5.0]: 1.5 1.5 5.1 6.2
After adding sum(min+max) to each: 9.2 9.2 12.8 13.9

========== TASK 2: multiset ==========
Initial multiset: (1,2) (1,2) (3,4) (3,4) (5,6) (7,8)
After adding average to end: (1,2) (1,2) (3,4) (3,4) (3,4) (5,6) (7,8)
After deleting elements in range (2,3) to (6,7): (1,2) (1,2) (7,8)
After adding sum(min+max) to each: (9,12) (9,12) (15,18)

========== TASK 3: ParameterizedList ==========
Initial list: 1.5 2.3 2.3 3.9 4.7 5.1
After adding average: 1.5 2.3 2.3 3.3 3.9 4.7 5.1
After deleting in (2.0,5.0): 1.5 5.1
After adding sum(min+max): 8.1 11.7

========== TASK 3: ParameterizedList ==========
Initial list: (1,2) (1,2) (3,4) (5,6) (7,8)
After adding average: (1,2) (1,2) (3,4) (3,4) (5,6) (7,8)
After deleting in range: (1,2) (1,2) (7,8)
After adding sum(min+max): (9,12) (9,12) (15,18)
```

6. Ответы на вопросы

```
Ассоциативный контейнер — это контейнер, который хранит элементы в отсортированном порядке и обеспечивает
быстрый доступ к элементам по ключу, а не по позиции. В отличие от последовательных контейнеров,
ассоциативные контейнеры автоматически упорядочивают элементы согласно заданному критерию.

Ассоциативные контейнеры библиотеки STL: set, multiset, map, multimap. Также сюда можно отнести
unordered_set, unordered_multiset, unordered_map, unordered_multimap (неупорядоченные ассоциативные
контейнеры, основанные на хешировании).

Доступ к элементам ассоциативного контейнера осуществляется по ключу с помощью оператора [] (для map),
метода find(), методов lower_bound() и upper_bound(), а также с помощью итераторов, которые позволяют
обходить элементы в отсортированном порядке.
```

Примеры методов: insert() — добавление элемента; erase() — удаление по ключу или итератору; find() — поиск по ключу; count() — подсчёт элементов с заданным ключом; lower_bound() — первый элемент не меньше ключа; upper_bound() — первый элемент больше ключа; equal_range() — диапазон элементов с данным ключом.

Создать контейнер map можно следующими способами:
map m1; — пустой map с ключом int и значением string.
map m2 = {{1, "one"}, {2, "two"}}; — инициализация списком.
map m3(m2); — копирование.
map m4(m2.begin(), m2.end()); — диапазоном итераторов.

По умолчанию элементы в map упорядочены по ключу в порядке возрастания с помощью компаратора less (от меньшего к большему). Чтобы изменить порядок на обратный, нужно использовать компаратор greater: map> m;

Операции, определённые для map: оператор [] для доступа и вставки; insert() и emplace() для добавления; erase() для удаления; find() для поиска; at() для доступа с проверкой границ; size(), empty(), clear(); итераторные операции begin(), end(), rbegin(), rend(); swap() для обмена содержимым; count() для проверки наличия ключа.

Функция для добавления элементов в map с помощью make_pair():
```
void addPair(map& m, int key, const string& value) {
    m.insert(make_pair(key, value));
}
```

Функция для добавления элементов в map с помощью оператора []:
```
void addWithBracket(map& m, int key, const string& value) {
    m[key] = value;
}
```

Функция для печати map с помощью итератора:
```
void printWithIterator(const map& m) {
    for (auto it = m.begin(); it != m.end(); ++it) {
        cout << it->first << " -> " << it->second << endl;
    }
}
```
Или с использованием range-based for:
```
void printWithIterator(const map& m) {
    for (const auto& p : m) {
        cout << p.first << " -> " << p.second << endl;
    }
}
```

Функция для печати map с помощью оператора [] (не рекомендуется, так как оператор [] в map для константного доступа не подходит, а для неконстантного создаёт элементы). Корректный вариант с использованием [] только для существующих ключей:
```
void printWithBracket(map& m) {
    for (auto it = m.begin(); it != m.end(); ++it) {
        cout << it->first << " -> " << m[it->first] << endl;
    }
}
```

Отличия map и multimap: map хранит уникальные ключи (каждому ключу соответствует только одно значение), при вставке дублирующего ключа вставка игнорируется. multimap допускает хранение нескольких элементов с одинаковым ключом, предоставляет методы equal_range() для получения диапазона всех элементов с одним ключом, и не имеет оператора [].

Контейнер set — это ассоциативный контейнер, который хранит уникальные отсортированные ключи без ассоциированных с ними значений. Каждый элемент является одновременно и ключом, и значением.

Отличия map и set: map хранит пары ключ-значение, обеспечивая отображение из ключа в данные; set хранит только ключи (одиночные значения). К элементам map обращаются через first и second, к элементам set — непосредственно. map позволяет модифицировать значение, ассоциированное с ключом, но не сам ключ; set не позволяет модифицировать элементы, так как они сами являются ключами.

Создать контейнер set можно следующим образом:
set s1; — пустой set целых чисел.
set s2 = {1, 2, 3, 4}; — инициализация списком.
set s3(s2); — копирование.
set s4(s2.begin(), s2.end()); — диапазоном итераторов.
set> s5; — set строк в обратном порядке.

По умолчанию элементы в set упорядочены по возрастанию с помощью less. Чтобы изменить порядок на обратный, нужно указать компаратор greater: set> s;

Операции, определённые для set: insert() для добавления; erase() для удаления; find() для поиска; count() для проверки наличия; lower_bound(), upper_bound(), equal_range() для поиска диапазонов; size(), empty(), clear(); итераторные операции begin(), end(), rbegin(), rend(); swap().

Функция для добавления элементов в set:
```
void addToSet(set& s, int value) {
```

```
    s.insert(value);
}
Для проверки успешности вставки:
bool addToSet(set& s, int value) {
    return s.insert(value).second;
}
```